

Csharp上位机实战：多线程编程（线程同步，事件触发，资源共享）

上位机作为人机交互的核心组件，承担着数据采集、状态监控、命令下发等重要任务。为了实现高效稳定的系统运行，掌握 **C# 多线程编程** 是每一位上位机开发必须具备的技能。

关键多线程技术：

- 线程同步（SynchronizationContext）
- 事件触发机制（Event）
- 信号量（ManualResetEvent）
- 互斥锁（Mutex）
- 共享内存（MemoryCache）
- 消息队列（扩展方向）

1.线程同步：子线程安全操作主线程控件

WinForm中的UI控件只能有创建它们的线程访问。如果试图从子线程直接修改控件，会抛出异常：“线程间操作无效”。

解决方案：

使用 `SynchronizationContext` 将子线程的操作调度到主线程执行。

```
private SynchronizationContext context; context =  
SynchronizationContext.Current; context.Send(ProgressUI, i); // 安全地更新UI
```

替代方法：

也可以使用 `Control.Invoke` 或 `BeginInvoke` 实现类似功能：

```
textBox1.BeginInvoke((MethodInvoker)delegate { textBox1.Text = i.ToString();  
});
```

但 `SynchronizationContext` 更通用，适合封装进独立类库中复用。

2.事件触发：实现跨线程通信

使用自定义委托和事件，可以在不同线程之间传递信息，实现模块解耦。

```
public delegate void DoSth(object sender); public event DoSth OnMyEvent;  
  
OnMyEvent(i); // 触发事件
```

通过注册一个事件处理器 `ProgressEvent` 来响应来自子线程的事件通知：

```
private void ProgressEvenet(object sender)
{
    Console.WriteLine($"线程事件触发: {sender.ToString()}");
}
```

这种方式非常适合用于状态变更通知，任务完成回调等场景。

3.互斥锁（Mutex）：保护共享资源不被并发访问

当多个线程需要访问同一资源时，容易引发冲突。使用 Mutex 可以确保某一时刻只有一个线程能访问该资源。

```
Mutex mutex = new Mutex();
void ThdMutex(object obj)
{
    mutex.WaitOne(); // 获取锁
    Console.WriteLine($"互斥线程 {Thread.CurrentThread.ManagedThreadId} 正在执行任务");
    Thread.Sleep(1000);
    Console.WriteLine($"互斥线程 {Thread.CurrentThread.ManagedThreadId} 任务执行完毕");
    mutex.ReleaseMutex(); // 释放锁
}
```

适用于串口通信，PLC写入，硬件接口访问等资源独占性要求高的场景

4.信号量（ManualResetEvent）：实现线程控制

有时我们需要某个线程等待另一个线程完成后才能继续执行。这时可以使用 ManualResetEvent 来实现线程间的流程控制。

```
ManualResetEvent manualReset = new ManualResetEvent(false);
// A线程先执行
ThreadPool.QueueUserWorkItem(state =>
{
    Console.WriteLine("模拟信号量，A开始执行");
    Thread.Sleep(5000);
    manualReset.Set(); // 释放信号
    Console.WriteLine("模拟信号量，A执行完成"); });
// B线程等待A完成
ThreadPool.QueueUserWorkItem(state =>
{
    Console.WriteLine("模拟信号量，B等待执行");
    manualReset.Reset(); // 重置信号
}
```

```
manualReset.WaitOne(); // 阻塞直到收到信号
Console.WriteLine("模拟信号量，B执行完成"); });
```

- Set()：发送信号，允许其他线程继续。
- WaitOne()：阻塞当前线程，直到收到信号。

适用于顺序执行、依赖任务调度等场景。

5.共享内存：线程间高效的数据共享方式

使用缓存或静态变量实现线程间数据共享，示例封装了 MemoryCacheHelper 类来模拟共享内存行为。

```
ThreadPool.QueueUserWorkItem(state =>
{
MemoryCacheHelper.SetCache("10001", "123"); // 写入共享数据
});

ThreadPool.QueueUserWorkItem(state =>
{
    var val =MemoryCacheHelper.GetCache("10001"); // 读取共享数据
    Console.WriteLine($"共享内存: {val}");
});
```

MemoryCacheHelper 的完整实现如下：

```
using System;
using System.Runtime.Caching;
namespace UtilForm.Util
{
    public class MemoryCacheHelper
    {
        private static MemoryCache cache = MemoryCache.Default;
        public static object GetCache(string key)
        {
            return cache.Get(key);
        }
        public static void SetCache(string key, object obj, int timeout =
7200)
        {
            cache.Set(key, obj, DateTimeOffset.Now.AddSeconds(timeout));
        }
        public static void RemoveCache(string key) {
            cache.Remove(key);
        }
    }
}
```

```
        }  
    }  
}
```

注意：实际使用中应结合线程锁机制保证线程安全。

总结

本文围绕一个完整的 WinForm 示例展开，详细讲解了上位机开发中常见的线程管理技术：

技术	用途	推荐使用场景
<code>SynchronizationContext</code>	子线程更新UI	所有涉及UI更新的异步操作
自定义事件	跨线程通信	模块解耦、状态通知
<code>Mutex</code>	资源互斥访问	串口通信、PLC写入等
<code>ManualResetEvent</code>	流程控制	多步骤依赖任务
共享内存（缓存/静态类）	数据共享	多线程数据传递
消息队列（推荐扩展）	异步任务调度	数据采集、日志系统等

掌握这些多线程技术，不仅有助于提高上位机程序的性能和稳定性，也为后续学习更复杂的并发模型（如TPL、`async/await`）打下了坚实基础。

提示建议

- 在实际开发中，优先考虑使用 `Task Parallel Library (TPL)` 和 `async/await` 来简化并发编程。
- 对于复杂系统，可结合 **线程池 + 队列 + 锁机制** 构建任务调度引擎。
- 多线程编程需特别注意死锁、竞态条件等问题，建议使用调试工具辅助排查。